

Chapter 24: Single-Source Shortest-Paths

1) Given a weighted, directed graph $G = (V, E)$ with weight function $w: E \rightarrow \mathbb{R}$ mapping edges to real-valued weights. The weight of a path $p = \langle v_0, v_1, \dots, v_k \rangle$ is the sum of the weights of its constituent edges:

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$$

The "shortest-path weight" from u to v is defined by :-

$$\delta(u, v) = \begin{cases} \min \{ w(p) : u \rightsquigarrow v \}, & \text{if there is a path from } u \text{ to } v \\ \infty, & \text{otherwise} \end{cases}$$

→ Edge weights can be interpreted as metrics other than distances. They are often used to represent time, cost, penalties, loss or any other quantity that accumulates linearly along a path and that one wishes to minimize.

2) Variants

The single-source shortest-paths problem is: Given a graph $G = (V, E)$ we want to find a shortest path from a given source vertex s to every vertex $v \in V$. Many other problems can be solved by the algorithm for the single-source problem, including the following variants:-

i) Single-destination shortest-paths problem:

Find a shortest path to a given destination vertex t from each vertex v . By reversing the

direction of each edge in the graph, we can reduce this problem to a single-source problem.

ii) Single-pair shortest-path problem: Find ~~the~~ a shortest path from u to v for given vertices u and v . If we solve the single-source problem with source vertex u we solve this problem also. Moreover, no algorithms for this problem are known that run asymptotically faster than the best single-source algorithms in the worst case.

iii) All-pairs shortest-paths problem: Find a shortest path from u to v for every pair of vertices u and v .

3) Optimal substructure of a shortest path

Lemma (Subpaths of shortest paths are shortest paths)

Let $p = \langle v_1, v_2, \dots, v_k \rangle$ be a shortest path from vertex v_1 to vertex v_k . For any i and j such that $1 \leq i \leq j \leq k$, let $p_{ij} = \langle v_i, v_{i+1}, v_{i+2}, \dots, v_j \rangle$ be the subpath of p from vertex v_i to vertex v_j . Then p_{ij} is a shortest path from v_i to v_j .

4) Negative-weight edges

→ In some instances of the "SSSP" problem there may be some edges with negative weights. In these cases, for some vertices v , the shortest-path weights $\delta(s, v)$ may have a negative value.

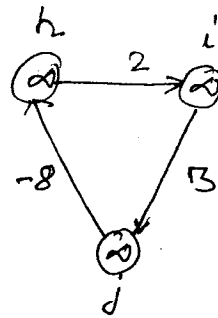
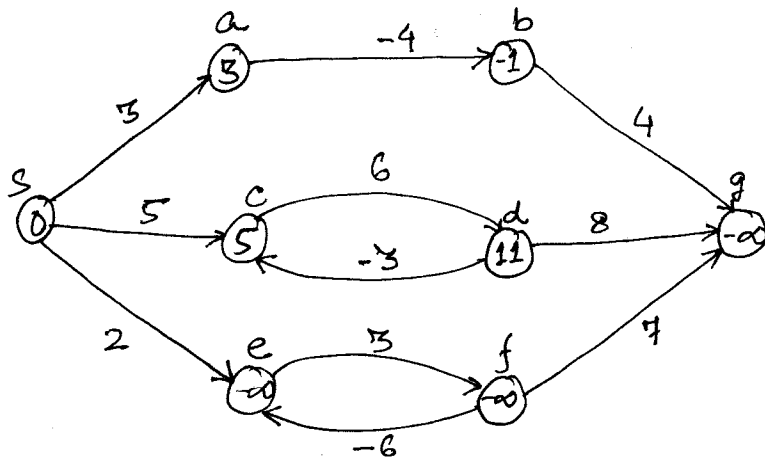
→ If the sum of weights of all edges along a cycle is negative, the cycle is said to be a

negative weight cycle.

→ If a graph does not contain any negative weight cycle reachable from the source, then for every vertex $v \in V$, the shortest path weight $\delta(s, v)$ remains well defined.

But if the graph contains a negative-weight cycle reachable from the source, then shortest-path weights are not well defined for those vertices which fall on the negative-weight cycle or for those vertices a path from the source to whom ~~contains~~ includes a negative-weight cycle. For those vertices we define $\delta(s, v) = -\infty$.

eg:



Shown within each vertex v is $\delta(s, v)$. Even though h, i, j lie on a negative weight cycle, their shortest-path weights are ∞ , because they are not reachable from the source.

5) Cycles:

A shortest path cannot contain a negative weight cycle. It cannot contain a positive-weight cycle either, since removing the cycle from the path produces a path with the same source and destination vertices

but with a longer path weight.

If a shortest path contains a 0-weight cycle, we can remove the 0-weight cycle to obtain a path of lesser no. of edges. Therefore, without loss of generality we can assume that shortest paths have no cycles.

Since any acyclic path in a graph can contain at most $|V|-1$ edges we can restrict our attention to shortest paths of at most $|V|-1$ edges.

6) Representing shortest paths:

Given a graph $G=(V,E)$ we maintain for every vertex $v \in V$ a predecessor $\pi[v]$ that is either another vertex or NIL.

The predecessor subgraph $G_\pi = (V_\pi, E_\pi)$ is induced by the π values, where:

$$V_\pi = \{v \in V : \pi[v] \neq \text{NIL}\} \cup \{s\}$$

$$E_\pi = \{(\pi[v], v) \in E : v \in V_\pi - \{s\}\}$$

A shortest-paths tree rooted at s is a directed subgraph $G' = (V', E')$ such that:-

- (i) V' is the set of vertices reachable from s in G ,
- (ii) G' forms a rooted tree with root s , and
- (iii) for all $v \in V'$, the unique simple path from s to v in G' is a shortest path from s to v in G .

→ shortest paths and shortest-paths trees may not be unique.

7) Relaxation:

For every vertex v we maintain an attribute $d[v]$, called a shortest-path estimate, which is an upper-bound on the weight of a shortest path from source s to v .

The following $\Theta(V)$ time procedure can be used to initialize the shortest-path estimates and predecessors:—

INITIALIZE_SINGLE-SOURCE(G, c)

- 1 for every vertex $v \in V[G]$
- 2 do $d[v] \leftarrow \infty$
- 3 $\pi[v] \leftarrow \text{NIL}$
- 4 $d[s] \leftarrow 0$

The process of relaxing an edge (u, v) consists of testing whether we can improve the shortest path to v bound so far by going through u , and if so, updating $d[v]$ and $\pi[v]$. The following code performs a relaxation step on edge (u, v) .

RELAX(u, v, w)

- 1 if $d[v] > d[u] + w(u, v)$
- 2 then $d[v] \leftarrow d[u] + w(u, v)$
- 3 $\pi[v] \leftarrow u$

→ It may seem strange that the term relaxation is used for an operation that actually tightens an upper bound. If there is an edge (u, v) , the relaxation step actually ~~tightens the~~ relaxes the constraint

$d[v] \leq d[u] + w(u,v)$, which by the triangle inequality must be satisfied if $d[u] = \delta(s,u)$ and $d[v] = \delta(s,v)$. If $d[v] \leq d[u] + w(u,v)$, there is no pressure to satisfy this constraint, so the constraint is relaxed.

→ Each ssSP algorithm calls INITIALIZE-SINGLE-SOURCE and then repeatedly relaxes edges. Relaxation is the only means by which shortest-path estimates and predecessors change. Different algorithms differ in how many times they relax each edge and in the order in which they relax edges.

The Dijkstra's algorithm and the shortest-paths algorithm for directed acyclic graphs relax each edge exactly once. The Bellman-Ford algorithm relaxes each edge many times.

8)

→ **Triangle inequality**: For any edge $(u,v) \in E$, we have $\delta(s,v) \leq \delta(s,u) + w(u,v)$.

→ **Upper-bound property**: We always have $d[v] \geq \delta(s,v)$ for all vertices $v \in V$, and once $d[v]$ achieves the value $\delta(s,v)$ it never changes.

→ **No-path property**: If there is no path from s to v , we always have $d[v] = \delta(s,v) = \infty$.

→ **Convergence property**: If $s \rightsquigarrow u \rightarrow v$ is a shortest path in G for some $u, v \in V$, and if $d[u] = \delta(s,u)$ at any time prior to relaxing the edge (u,v) , then $d[v] = \delta(s,v)$ at all times afterward.

→ **Path-relaxation property:** If $p = \langle v_0, v_1, \dots, v_k \rangle$ is a shortest path from $s = v_0$ to v_k , and the edges of p are relaxed in the order $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$, then $d[v_k] = \delta(s, v_k)$. This property holds regardless of any other relaxation steps that occur, even if they are interleaved with relaxations of the edges of p .

→ **Predecessor-subgraph property:**

Once $d[v] = \delta(s, v)$ for all $v \in V$, the predecessor subgraph is a shortest-paths tree rooted at s .

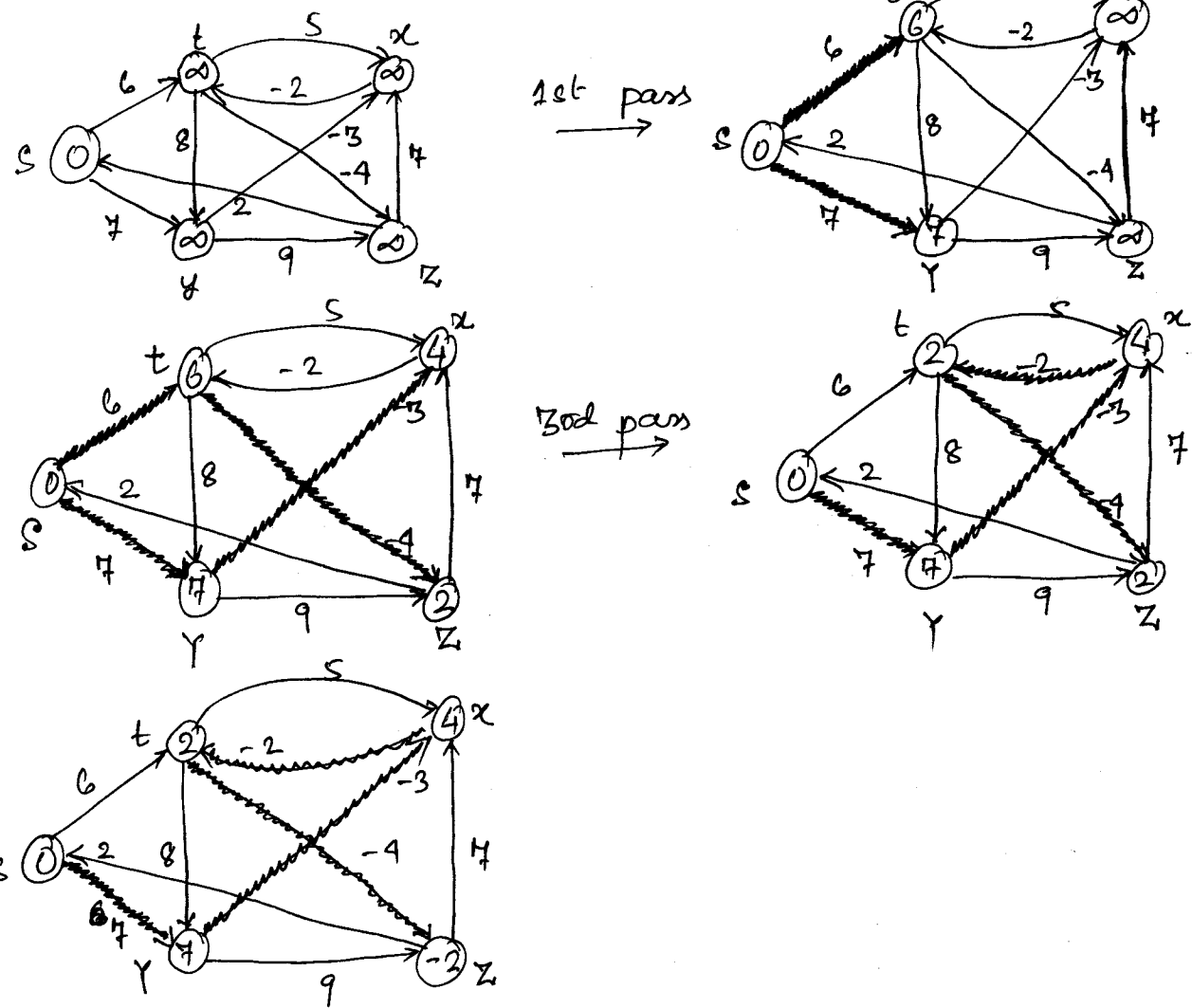
1) The Bellman-Ford algorithm

The Bellman-Ford algorithm solves the single-source shortest-paths problem in the general case in which edge-weights may be negative. The Bellman-Ford algorithm returns a boolean value indicating whether or not there is a negative weight cycle reachable from the source. If there is such a cycle, the algorithm returns FALSE indicating that no solution exists.

BELLMAN-FORD (G, w, s)

```
1 INITIALIZE-SINGLE-SOURCE( $G, s$ )
2 for  $i \leftarrow 1$  to  $|V[G]| - 1$ 
3   do for each edge  $(u, v) \in E[G]$ 
4     do relax( $u, v, w$ )
5   for each edge  $(u, v) \in E[G]$ 
6     do if  $d[v] > d[u] + w(u, v)$ 
7       then return FALSE
8 return TRUE
```

eg: d values are shown within the vertices. Shaded edges indicate predecessor values. Each pass relaxes the edges in the order $(t,x), (t,y), (t,z), (x,t), (y,x), (y,z), (z,x), (z,s), (s,t), (s,y)$.



The algorithm makes $|V|-1$ passes over the edges of the graph. Each pass consists of relaxing every edge of the graph once.

The Bellman-Ford algorithm runs in time $O(VE)$.

Lemma

Let G be a weighted directed graph and assume that G does not contain any negative-weight cycle reachable from the source. Then, after the $|V|-1$ iterations of the ~~for~~ loop of lines 2-4 of BELLMAN-FORD, we have $d[v] = \delta(s, v)$ for all vertices v that are reachable from s .

Proof: We prove the lemma by using the path-relaxation property. Consider any vertex v that is reachable from s , and let $p = \langle v_0, v_1, v_2, \dots, v_k \rangle$ where $v_0 = s$ and $v_k = v$ be any acyclic shortest path from s to v . Path p has at most $|V|-1$ edges and so $k \leq |V|-1$. Each of the $|V|-1$ iterations of the ~~for~~ loop relaxes all E edges. The i th iteration of the ~~for~~ loop ~~also~~ relaxes the edge (v_{i-1}, v_i) ^{of path p} among others. Thus, we see that there is a relaxation of every edge of path p in order from the 1st edge to the last.

Therefore, by path-relaxation property we have $d[v] = \delta(s, v)$ or $d[v_k] = \delta(s, v_k)$.

Theorem (Correctness of the BELLMAN-FORD algorithm):

Let BELLMAN-FORD algorithm be run on a weighted, directed graph G . If G does not contain any negative-weight cycle reachable from s , then the algorithm returns TRUE and we have $d[v] = \delta(s, v)$ for all vertices $v \in V$, and the predecessor subgraph G_π is a shortest-paths tree rooted at s . If G does contain a negative-weight cycle

cycle reachable from s , then the algorithm returns FALSE.

Proof: Suppose the graph G does not contain any negative-weight cycles reachable from the source.

The previous Lemma can be used to prove that at termination $d[v] = \delta(s, v)$ for all vertices $v \in V$.

If v is not reachable from s , the no-path property implies that $d[v] = \delta(s, v) = \infty$. The predecessor-subgraph property, along with the claim implies that G_n is a shortest-paths tree rooted at s .

Now we use the claim ^{to show} that BELLMAN-FORD returns TRUE. A termination, we have for every edge $(u, v) \in E$:-

$$\begin{aligned} d[v] = \delta(s, v) &\leq \delta(s, u) + w(u, v) \quad \left[\begin{array}{c} \text{By} \\ \text{triangle} \\ \text{inequality} \end{array} \right] \\ &= d[u] + w(u, v) \end{aligned}$$

Hence, none of the tests in line 6 causes

BELLMAN-FORD to return FALSE. It therefore returns TRUE.

Now suppose the graph G contains a negative-weight cycle reachable from source s . Let this cycle be $C = \langle v_0, v_1, \dots, v_k \rangle$ where $v_0 = v_k$:-

Then $\sum_{i=1}^k w(v_{i-1}, v_i) < 0$

Assume for the purpose of contradiction that the BELLMAN-FORD algorithm returns TRUE.

Thus, ~~$d[v_{i-1}]$~~

$$d[v_i] \leq d[v_{i-1}] + W(v_{i-1}, v_i) \text{ for } i=1, 2, \dots, K$$

Summing up the inequality around the cycle gives us:—

$$\sum_{i=1}^K d[v_i] \leq \sum_{i=1}^K d[v_{i-1}] + \sum_{i=1}^K W(v_{i-1}, v_i)$$

Noting that $v_0 = v_K$ and since every $d[v_i]$ is finite (since BELLMAN-FORD returns true) we have—

$$\sum_{i=1}^K d[v_i] = \sum_{i=1}^K d[v_{i-1}]$$

Thus,
$$0 \leq \sum_{i=1}^K W(v_{i-1}, v_i)$$

Which contradicts the previous inequality. Thus, we prove by contradiction that if the graph G does not contain any negative-weight cycle reachable from s , BELLMAN-FORD returns TRUE, ~~and~~ false otherwise.

10) Single-source shortest paths in directed acyclic graphs:

DAG-SHORTEST-PATHS(G, w, s)

- 1 topologically sort the vertices of G
- 2 INITIALIZE-SINGLE-SOURCE(G, s)
- 3 for each vertex u taken in topologically sorted order
- 4 do for each vertex $v \in \text{Adj}[u]$
- 5 do RELAX(u, v, w)

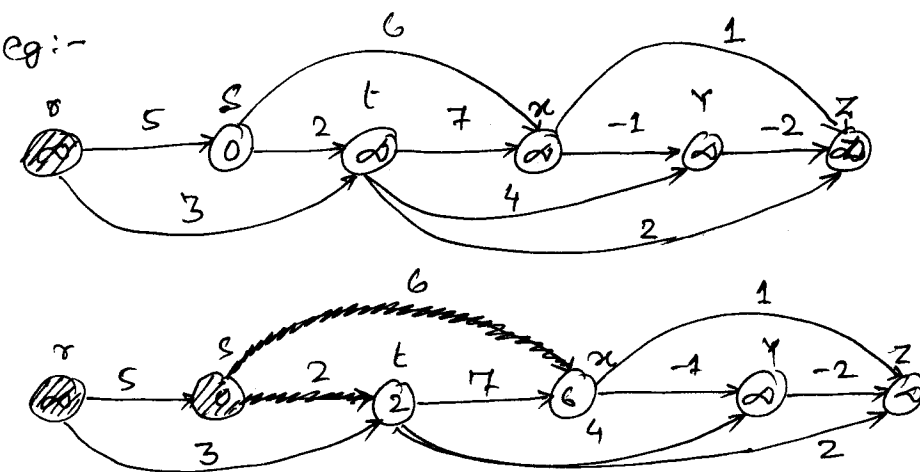
→ If there is a path from vertex u to vertex v then u precedes v in a topological sort. By relaxing the edges of a weighted dag according to a topological sort of its vertices, we can compute shortest paths from a single source in just a single pass.

→ We make just one pass over the vertices in a topologically sorted order. As we process each vertex, we relax each edge that leaves the vertex.

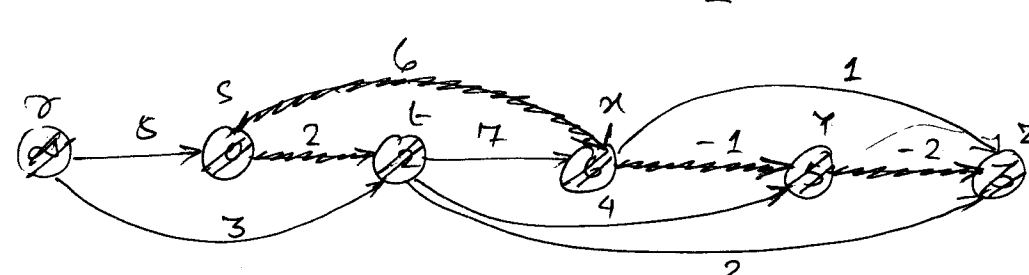
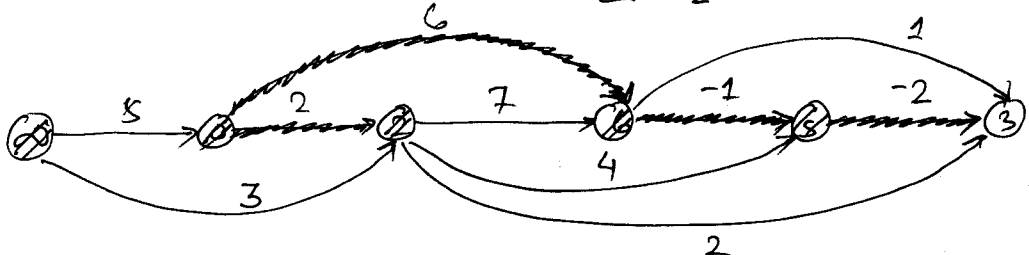
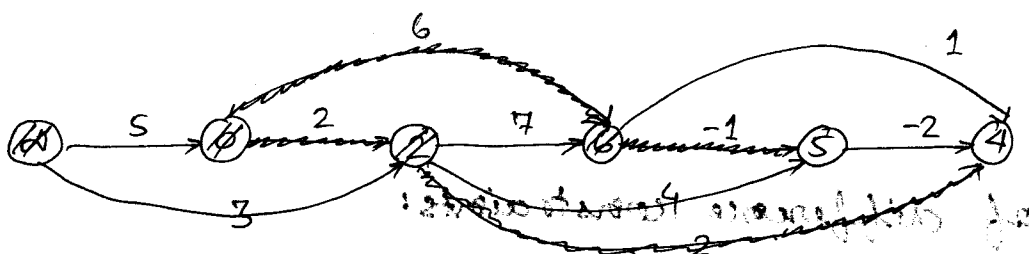
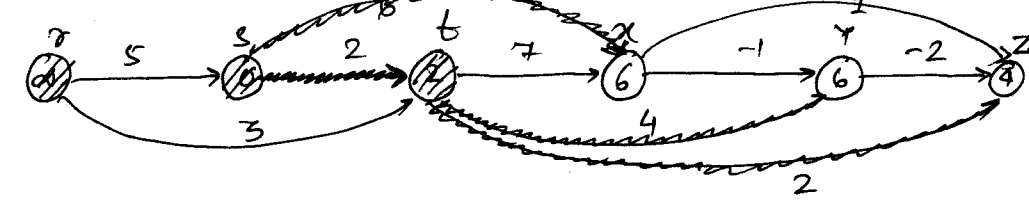
→ Shortest paths are always well defined in a dag, since even if there are negative-weight edges, no negative-weight cycles can exist.

→ Topological sort of vertices can be performed in $\Theta(V+E)$ time. INITIALIZE-SINGLE-SOURCE takes $\Theta(V)$ time. For every vertex, the edges that leave the vertex are examined exactly once. Thus, there are a total of $|E|$ iterations of the inner for loop of lines 4-5. (We have used aggregate analysis here.) Each iteration of inner for loop takes $\Theta(1)$ time to relax an edge. Thus, the total run time is $\Theta(V+E)$.

eg:-



Shaded edges represent processed predecessors. Source is s .



11) Difference constraints and shortest paths

→ Linear Programming: In the general "linear-programming problem" we wish to find a vector x of n elements that maximizes the objective function $\sum_{i=1}^n C_i x_i$ subject to m constraints in n unknowns given by : $Ax \leq b$

where, A is a $m \times n$ matrix

b is a vector of length m

→ Sometimes we don't care about the objective function. We just wish to find any feasible solution that is, any vector x that satisfies $Ax \leq b$, or to determine that no feasible solution exists. Such a problem is called a feasibility problem.

→ Here we investigate a special case of linear programming (feasibility problem) that can be reduced to finding shortest paths from a single source. The single-source shortest-paths problem that results can then be solved using BELLMAN-FORD algorithm.

Systems of difference constraints:

In a system of difference constraints, you have a set of m "difference constraints" in n unknowns. Each difference constraint is an inequality of the form $x_j - x_i \leq b_k$ $1 \leq i, j \leq n$ and $1 \leq k \leq m$.

Thus, every row of the matrix A of $Ax \leq b$ contains a single 1 and a single -1 and remaining 0's.

For example: -

$$\begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 & -1 \\ -1 & 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & -1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix} \leq \begin{pmatrix} 0 \\ -1 \\ 1 \\ 4 \\ -1 \\ -3 \\ -3 \end{pmatrix}$$

It can also be written as: -

$$x_1 - x_2 \leq 0$$

$$x_1 - x_5 \leq -1$$

⋮

$$x_5 - x_4 \leq -3$$

2 feasible solutions to the above problem are ~~arbitrary~~

$x = (-5, -3, 0, -1, -4)$ and $x' = (0, 2, 5, 4, 1)$. Each component of x' is 5 larger than the corresponding component of x .

Lemma:

Let $x = (x_1, x_2, \dots, x_n)$ be a solution to a system $Ax \leq b$ of difference constraints and let d be any constant. Then $x + d = (x_1 + d, x_2 + d, \dots, x_n + d)$ is also a solution to $Ax \leq b$.

Proof: For each x_j and x_i we have $(x_j + d) - (x_i + d) = x_j - x_i$. Thus, if x satisfies $Ax \leq b$, so does $x + d$.

Constraint graphs

Given a system $Ax \leq b$ of difference constraints, the corresponding constraint graph is a weighted, directed graph $G = (V, E)$, where

the vertex set V consists of a vertex v_i for each unknown x_i , plus an additional vertex v_0 .
i.e. $V = \{v_0, v_1, v_2, \dots, v_n\}$

and

the edge set E consists of an edge for each difference constraint, plus an edge (v_0, v_i) for each unknown v_i . Corresponding to a constraint $x_j - x_i \leq b_k$ you have an edge leaving v_i and entering v_j and weighting b_k i.e. $w(v_i, v_j) = b_k$. Height of each edge leaving v_0 is 0. i.e.

$$E = \{ (v_i, v_j) : x_j - x_i \leq b_k \text{ is a constraint} \} \cup \{ (v_0, v_1), (v_0, v_2), \dots, (v_0, v_n) \}$$

Theorem

Given a system $Ax \leq b$ of difference constraints, let G be the corresponding constraint graph. If G does not contain any negative weight cycle then $x = (s(v_0, v_1), s(v_0, v_2), \dots, s(v_0, v_n))$ is a feasible solution for the system. If G contains a negative weight cycle there is no feasible solution for the system.

Proof: Every negative weight cycle is reachable from the source vertex v_0 , since every other vertex is reachable from the source vertex v_0 .

Suppose the constraint graph does not contain any negative weight cycle. For any edge $(v_i, v_j) \in E$ we have by triangle inequality :-

$$s(v_0, v_j) \leq s(v_0, v_i) + w(v_i, v_j)$$

$$\Rightarrow s(v_0, v_j) - s(v_0, v_i) \leq w(v_i, v_j)$$

Letting $x_j = s(v_0, v_j)$, $x_i = s(v_0, v_i)$ and $b_k = w(v_i, v_j)$

satisfies the difference constraint $x_j - x_i \leq b_k$.

Thus, if we let $x = (s(v_0, v_1), s(v_0, v_2), \dots, s(v_0, v_n))$ we have a feasible solution to $Ax \leq b$.

If the constraint graph contains a negative-weight cycle, let the negative weight cycle be $C = \langle v_i, v_{i+1}, v_{i+2}, \dots, v_j \rangle$ where $v_i = v_j \neq v_0$ (since no edge enters v_0). Then,

$$\sum_{k=i}^{j-1} w(v_k, v_{k+1}) < 0$$

Suppose for the purpose of contradiction, that we have a solution $x = (x_1, \dots, x_j)$. Then:-

$$x_{i+1} - x_i \leq w(x_i, x_{i+1})$$

$$x_{i+2} - x_{i+1} \leq w(x_{i+1}, x_{i+2})$$

$\vdots$

$$x_j - x_{j-1} \leq w(x_{j-1}, x_j)$$

$$\sum 0 \leq w(c) \text{ which contradicts the}$$

previous inequality. Thus, by contradiction, we prove that if G contains a negative weight ~~edge~~^{cycle}, there is no feasible solution for the system.

→ We can use the BELLMAN-FORD algorithm to solve a system of difference constraints. If the BELLMAN-FORD algorithm returns TRUE, then the shortest-paths weights give a feasible solution to the system. If the BELLMAN-FORD algorithm returns FALSE, there is no feasible solution to the system of difference constraints.

→ A system of difference constraints with m constraints and n unknowns produces a graph with $(n+1)$ vertices and $(m+n)$ edges. Using the BELLMAN-FORD algorithm we can solve the system in $O((n+1)(m+n)) = O(n^2 + mn)$ time.